

Project 1 (DSP48A1)

- First Section: RTL Code

```
1  module DSP48A1 #(
2      parameter A0REG = 0,
3      parameter A1REG = 1,
4      parameter B0REG = 0,
5      parameter B1REG = 1,
6      parameter CREG = 1,
7      parameter DREG = 1,
8      parameter MREG = 1,
9      parameter PREG = 1,
10     parameter CARRYINREG = 1,
11     parameter CARRYOUTREG = 1,
12     parameter OPMODEREG = 1,
13
14     // Configuration parameters
15     parameter CARRYINSEL = "OPMODE5", // "CARRYIN" or "OPMODE5"
16     parameter B_INPUT = "DIRECT", // "DIRECT" or "CASCADE"
17     parameter RSTTYPE = "SYNC" // "SYNC" or "ASYNC"
18
19 ) (
20     // Clock
21     input CLK,
22
23     //Data Inputs
24
25     input [17:0] A,
26     input [17:0] B,
27     input [47:0] C,
28     input [17:0] D,
29     input CARRYIN,
30
```

```
31 | // Control inputs
32 | input [7:0] OPMODE,
33 |
34 | // Clock enables
35 | input CEA,
36 | input CEB,
37 | input CEC,
38 | input CECARRYIN,
39 | input CED,
40 | input CEM,
41 | input CEOPMODE,
42 | input CEP,
43 |
44 | // Resets
45 | input RSTA,
46 | input RSTB,
47 | input RSTC,
48 | input RSTCARRYIN,
49 | input RSTD,
50 | input RSTM,
51 | input RSTOPMODE,
52 | input RSTP,
53 |
54 | // Cascade inputs
55 | input [17:0] BCIN,
56 | input [47:0] PCIN,
57 |
58 | // Data outputs
59 | output [35:0] M,
60 | output [47:0] P,
61 | output CARRYOUT,
62 | output CARRYOUTF,
63 |
64 | // Cascade outputs
65 | output [17:0] BCOUT,
66 | output [47:0] PCOUT
67 |
68 | );
```

```
70 // Internal registers and wires
71 reg [17:0] A0_reg, A1_reg;
72 reg [17:0] B0_reg, B1_reg;
73 reg [47:0] C_reg;
74 reg [17:0] D_reg;
75 reg [7:0] OPMODE_reg;
76 reg CARRYIN_reg;
77 reg [35:0] M_reg;
78 reg [47:0] P_reg;
79 reg CARRYOUT_reg;
80
81 wire [17:0] A0_out, A1_out;
82 wire [17:0] B0_out, B1_out;
83 wire [47:0] C_out;
84 wire [17:0] D_out;
85 wire [7:0] OPMODE_out;
86 wire CARRYIN_out;
87 wire [35:0] M_out;
88 wire [47:0] P_out;
89 wire CARRYOUT_out;
90
91 wire [17:0] B_input_mux;
92 wire CARRYIN_mux;
93 wire [17:0] pre_adder_out;
94 wire [35:0] multiplier_out;
95 wire [47:0] X_mux, Z_mux;
96 wire [47:0] post_adder_out;
97 wire post_adder_carryout;
98
99
100
```

```

101 // Reset generation based on RSTTYPE
102 wire rst_a, rst_b, rst_c, rst_d, rst_m, rst_p, rst_carryin, rst_opmode;
103 generate
104     if (RSTTYPE == "ASYNC") begin
105         assign rst_a = RSTA;
106         assign rst_b = RSTB;
107         assign rst_c = RSTC;
108         assign rst_d = RSTD;
109         assign rst_m = RSTM;
110         assign rst_p = RSTP;
111         assign rst_carryin = RSTCARRYIN;
112         assign rst_opmode = RSTOPMODE;
113     end else begin
114         assign rst_a = 1'b0;
115         assign rst_b = 1'b0;
116         assign rst_c = 1'b0;
117         assign rst_d = 1'b0;
118         assign rst_m = 1'b0;
119         assign rst_p = 1'b0;
120         assign rst_carryin = 1'b0;
121         assign rst_opmode = 1'b0;
122     end
123 endgenerate
124
125 //B Input Multiplexer
126 generate
127     if (B_INPUT == "DIRECT")begin
128         assign B_input_mux = B;
129     end else if (B_INPUT == "CASCADE") begin
130         assign B_input_mux = BCIN;
131     end else begin
132         assign B_input_mux = 18'b0;
133     end
134 endgenerate

```

```

137 //CarryIn Multiplexer
138 generate
139     if (CARRYINSEL == "CARRYIN") begin
140         assign CARRYIN_mux = CARRYIN_out;
141     end else if (CARRYINSEL == "OPMODE5") begin
142         assign CARRYIN_mux = OPMODE_out[5];
143     end else begin
144         assign CARRYIN_mux = 1'b0;
145     end
146 endgenerate

147
148 // OPMODE register
149 generate
150     if (OPMODEREG == 1) begin
151         always @(posedge CLK or posedge RSTOPMODE) begin
152             if (rst_opmode)
153                 OPMODE_reg <= 8'b0;
154             else if (RSTTYPE == "SYNC" && RSTOPMODE)
155                 OPMODE_reg <= 8'b0;
156             else if (CEOPMODE)
157                 OPMODE_reg <= OPMODE;
158             end
159             assign OPMODE_out = OPMODE_reg;
160         end else begin
161             assign OPMODE_out = OPMODE;
162         end
163     endgenerate

```

```

165 // A pipeline registers
166 generate
167     if (A0REG == 1) begin
168
169         always @(posedge CLK or posedge rst_a) begin
170             if (rst_a)
171                 A0_reg <= 18'b0;
172             else if (RSTTYPE == "SYNC" && RSTA)
173                 A0_reg <= 18'b0;
174             else if (CEA)
175                 A0_reg <= A;
176             end
177
178             assign A0_out = A0_reg;
179
180         end else begin
181             assign A0_out = A;
182         end
183
184         if (A1REG == 1) begin
185
186             always @(posedge CLK or posedge RSTA) begin
187                 if (rst_a)
188                     A1_reg <= 18'b0;
189                 else if (RSTTYPE == "SYNC" && RSTA)
190                     A1_reg <= 18'b0;
191                 else if (CEA)
192                     A1_reg <= A0_out;
193                 end
194
195                 assign A1_out = A1_reg;
196
197             end else begin
198                 assign A1_out = A0_out;
199             end
200         endgenerate
201
202

```

```

203 // Pre-adder/subtractor
204 assign pre_adder_out = OPMODE_out[6] ? (D_out - B0_out) : (D_out + B0_out);
205
206
207 // B pipeline registers
208 generate
209     if (B0REG == 1) begin
210
211         always @(posedge CLK or posedge RSTB) begin
212             if (rst_b)
213                 B0_reg <= 18'b0;
214             else if (RSTTYPE == "SYNC" && RSTB)
215                 B0_reg <= 18'b0;
216             else if (CEB)
217                 B0_reg <= B_input_mux;
218             end
219
220             assign B0_out = B0_reg;
221
222         end else begin
223             assign B0_out = B_input_mux;
224         end
225
226     if (B1REG == 1) begin
227
228         always @(posedge CLK or posedge RSTB) begin
229             if (rst_b)
230                 B1_reg <= 18'b0;
231             else if (RSTTYPE == "SYNC" && RSTB)
232                 B1_reg <= 18'b0;
233             else if (CEB)
234                 B1_reg <= (OPMODE_out[4]) ? pre_adder_out : B0_out;
235             end
236
237             assign B1_out = B1_reg;
238
239         end else begin
240             assign B1_out = B0_out;
241         end
242     endgenerate
243
244

```

```

245 // C register
246 generate
247     if(CREG == 1) begin
248
249         always @(posedge CLK or posedge RSTC) begin
250             if(rst_c)
251                 C_reg <= 48'b0;
252             else if (RSTTYPE == "SYNC" && RSTC)
253                 C_reg <= 48'b0;
254             else if (CEC)
255                 C_reg <= C;
256             end
257
258             assign C_out = C_reg;
259
260         end else begin
261             assign C_out = C;
262         end
263     endgenerate
264
265 // D Register
266 generate
267     if(DREG == 1) begin
268
269         always @(posedge CLK or posedge RSTD) begin
270             if(rst_d)
271                 D_reg <= 18'b0;
272             else if (RSTTYPE == "SYNC" && RSTD)
273                 D_reg <= 18'b0;
274             else if (CED)
275                 D_reg <= D;
276             end
277
278             assign D_out = D_reg;
279
280         end else begin
281             assign D_out = D;
282         end
283     end
284 endgenerate
285
286
287 //CarryIn Register
288 generate
289     if (CARRYINREG == 1) begin
290         always @(posedge CLK or posedge RSTCARRYIN) begin
291             if (rst_carryin)
292                 CARRYIN_reg <= 1'b0;
293             else if (RSTTYPE == "SYNC" && RSTCARRYIN)
294                 CARRYIN_reg <= 1'b0;
295             else if (CECARRYIN)
296                 CARRYIN_reg <= CARRYIN;
297             end
298             assign CARRYIN_out = CARRYIN_reg;
299         end else begin
300             assign CARRYIN_out = CARRYIN;
301         end
302     end
303 endgenerate
304
305

```

```

306 // Multiplier input selection and multiplication
307 wire [17:0] mult_a = A1_out;
308 wire [17:0] mult_b = OPMODE_out[4] ? pre_adder_out : B1_out;
309 assign multiplier_out = mult_a * mult_b;
310
311
312 // M register
313 generate
314     if (MREG == 1) begin
315         always @(posedge CLK or posedge RSTM) begin
316             if (rst_m)
317                 M_reg <= 36'b0;
318             else if (RSTTYPE == "SYNC" && RSTM)
319                 M_reg <= 36'b0;
320             else if (CEM)
321                 M_reg <= multiplier_out;
322             end
323             assign M_out = M_reg;
324         end else begin
325             assign M_out = multiplier_out;
326         end
327     endgenerate

```

```

329 // X multiplexer (48-bit)
330 reg [47:0] X_mux_out;
331 always @(*) begin
332     case (OPMODE_out[1:0])
333         2'b00: X_mux_out = 48'b0;
334         2'b01: X_mux_out = {{12{M_out[35]}}, M_out}; // Sign extend M
335         2'b10: X_mux_out = P_out;
336         2'b11: X_mux_out = {D_out[11:0], A1_out, B1_out};
337         default: X_mux_out = 48'b0;
338     endcase
339 end
340 assign X_mux = X_mux_out;
341
342 // Z multiplexer (48-bit)
343 reg [47:0] Z_mux_out;
344 always @(*) begin
345     case (OPMODE_out[3:2])
346         2'b00: Z_mux_out = 48'b0;
347         2'b01: Z_mux_out = PCIN;
348         2'b10: Z_mux_out = P_out;
349         2'b11: Z_mux_out = C_out;
350         default: Z_mux_out = 48'b0;
351     endcase
352 end
353 assign Z_mux = Z_mux_out;
354

```

```

355 // Post-adder/subtractor
356 assign {post_adder_carryout, post_adder_out} =
357     OPMODE_out[7] ? (Z_mux - (X_mux + CARRYIN_mux)) :
358     (Z_mux + X_mux + CARRYIN_mux);
359
360 // P register
361 generate
362     if (PREG == 1) begin
363         always @(posedge CLK or posedge RSTP) begin
364             if (rst_p)
365                 P_reg <= 48'b0;
366             else if (RSTTYPE == "SYNC" && RSTP)
367                 P_reg <= 48'b0;
368             else if (CEP)
369                 P_reg <= post_adder_out;
370         end
371         assign P_out = P_reg;
372     end else begin
373         assign P_out = post_adder_out;
374     end
375 endgenerate
376
377 // CARRYOUT register
378 generate
379     if (CARRYOUTREG == 1) begin
380         always @(posedge CLK or posedge RSTCARRYIN) begin
381             if (rst_carryin)
382                 CARRYOUT_reg <= 1'b0;
383             else if (RSTTYPE == "SYNC" && RSTCARRYIN)
384                 CARRYOUT_reg <= 1'b0;
385             else if (CECARRYIN)
386                 CARRYOUT_reg <= post_adder_carryout;
387         end
388         assign CARRYOUT_out = CARRYOUT_reg;
389     end else begin
390         assign CARRYOUT_out = post_adder_carryout;
391     end
392 endgenerate
393
394
395 // Output assignments
396 assign M = M_out;
397 assign P = P_out;
398 assign CARRYOUT = CARRYOUT_out;
399 assign CARRYOUTF = CARRYOUT_out;
400 assign BCOUT = B1_out;
401 assign PCOUT = P_out;
402
403
404
405
406 endmodule

```

● Second Section: Testbench

```
1  module DSP48A1_tb();
2
3  // Test signals
4      reg CLK;
5      reg [17:0] A, B, D;
6      reg [47:0] C;
7      reg CARRYIN;
8      reg [7:0] OPMODE;
9      reg CEA, CEB, CEC, CECARRYIN, CED, CEM, CEOPMODE, CEP;
10     reg RSTA, RSTB, RSTC, RSTCARRYIN, RSTD, RSTM, RSTOPMODE, RSTP;
11     reg [17:0] BCIN;
12     reg [47:0] PCIN;
13
14     wire [35:0] M;
15     wire [47:0] P;
16     wire CARRYOUT, CARRYOUTF;
17     wire [17:0] BCOUT;
18     wire [47:0] PCOUT;
19
20     // Test tracking variables
21     integer test_num = 0;
22     integer pass_count = 0;
23     integer fail_count = 0;
24     reg [47:0] previous_P;
25     reg previous_CARRYOUT;
26
27
28     // Instantiate DUT with specified parameters
29     DSP48A1 #(
30         .A0REG(0), .A1REG(1), .B0REG(0), .B1REG(1),
31         .CREG(1), .DREG(1), .MREG(1), .PREG(1),
32         .CARRYINREG(1), .CARRYOUTREG(1), .OPMODEREG(1),
33         .CARRYINSEL("OPMODE5"), .B_INPUT("DIRECT"), .RSTTYPE("SYNC")
34     ) dut (
35         .CLK(CLK),
36         .A(A), .B(B), .C(C), .D(D), .CARRYIN(CARRYIN),
37         .OPMODE(OPMODE),
38         .CEA(CEA), .CEB(CEB), .CEC(CEC), .CECARRYIN(CECARRYIN),
39         .CED(CED), .CEM(CEM), .CEOPMODE(CEOPMODE), .CEP(CEP),
40         .RSTA(RSTA), .RSTB(RSTB), .RSTC(RSTC), .RSTCARRYIN(RSTCARRYIN),
41         .RSTD(RSTD), .RSTM(RSTM), .RSTOPMODE(RSTOPMODE), .RSTP(RSTP),
42         .BCIN(BCIN), .PCIN(PCIN),
43         .M(M), .P(P), .CARRYOUT(CARRYOUT), .CARRYOUTF(CARRYOUTF),
44         .BCOUT(BCOUT), .PCOUT(PCOUT)
45     );
```

1. Verifying Reset Operation

```
47 // Clock generation
48 initial begin
49     CLK = 0;
50     forever
51         #1 CLK = ~CLK;
52 end
53
54 // Task 1: Verify Reset Operation (Enhanced with Internal Signal Verification)
55 task test_reset_operation;
56     begin
57         $display("\nTest 1: Reset Operation Verification");
58         $display("=====");
59
60         // ADDITION: First load some data into registers to verify reset clears them
61         $display("Loading test data into internal registers...");
62         RSTA = 0; RSTB = 0; RSTC = 0; RSTCARRYIN = 0;
63         RSTD = 0; RSTM = 0; RSTOPMODE = 0; RSTP = 0;
64         CEA = 1; CEB = 1; CEC = 1; CECARRYIN = 1;
65         CED = 1; CEM = 1; CEOPMODE = 1; CEP = 1;
66
67         // Load known test patterns
68         A = 18'h3FFFF; B = 18'h2AAAA; C = 48'hAAAAAAAAAAAA;
69         D = 18'h15555; OPMODE = 8'hFF; CARRYIN = 1'b1;
70         BCIN = 18'h11111; PCIN = 48'h55555555555555;
71
72         // Wait for data to propagate
73         repeat(3) @(posedge CLK);
74
75         // ADDITION: Display internal register values before reset
76         $display("Internal registers BEFORE reset:");
77         $display(" A1_reg = 0x%h (A0REG=0, only A1 exists)", dut.A1_reg);
78         $display(" B1_reg = 0x%h (B0REG=0, only B1 exists)", dut.B1_reg);
79         $display(" C_reg = 0x%h", dut.C_reg);
80         $display(" D_reg = 0x%h", dut.D_reg);
81         $display(" OPMODE_reg = 0x%h", dut.OPMODE_reg);
82         $display(" M_reg = 0x%h, P_reg = 0x%h", dut.M_reg, dut.P_reg);
83
84         // Assert all active-high reset signals
85         RSTA = 1; RSTB = 1; RSTC = 1; RSTCARRYIN = 1;
86         RSTD = 1; RSTM = 1; RSTOPMODE = 1; RSTP = 1;
87
88         // Drive remaining inputs with arbitrary values
89         A = $random;
90         B = $random;
91         C = $random;
92         D = $random;
93         CARRYIN = $random;
94         OPMODE = $random;
95         BCIN = $random;
96         PCIN = $random;
```

```

98 // Wait for negative edge of clock
99 @(negedge CLK);
100
101 // ADDITION: Display internal register values after reset
102 $display("Internal registers AFTER reset:");
103 $display(" A1_reg = 0x%h", dut.A1_reg);
104 $display(" B1_reg = 0x%h", dut.B1_reg);
105 $display(" C_reg = 0x%h", dut.C_reg);
106 $display(" D_reg = 0x%h", dut.D_reg);
107 $display(" OPMODE_reg = 0x%h", dut.OPMODE_reg);
108 $display(" M_reg = 0x%h, P_reg = 0x%h", dut.M_reg, dut.P_reg);
109
110 // ADDITION: Verify internal registers are cleared
111 if (dut.A1_reg !== 18'h0) begin
112     $display("ERROR: A1_reg not cleared by RSTA (0x%h)", dut.A1_reg);
113     fail_count = fail_count + 1;
114 end else begin
115     $display("PASS: A1_reg cleared by RSTA");
116 end
117
118 if (dut.B1_reg !== 18'h0) begin
119     $display("ERROR: B1_reg not cleared by RSTB (0x%h)", dut.B1_reg);
120     fail_count = fail_count + 1;
121 end else begin
122     $display("PASS: B1_reg cleared by RSTB");
123 end
124
125 if (dut.C_reg !== 48'h0) begin
126     $display("ERROR: C_reg not cleared by RSTC (0x%h)", dut.C_reg);
127     fail_count = fail_count + 1;
128 end else begin
129     $display("PASS: C_reg cleared by RSTC");
130 end
131
132 if (dut.D_reg !== 18'h0) begin
133     $display("ERROR: D_reg not cleared by RSTD (0x%h)", dut.D_reg);
134     fail_count = fail_count + 1;
135 end else begin
136     $display("PASS: D_reg cleared by RSTD");
137 end
138
139 if (dut.OPMODE_reg !== 8'h0) begin
140     $display("ERROR: OPMODE_reg not cleared by RSTOPMODE (0x%h)", dut.OPMODE_reg);
141     fail_count = fail_count + 1;
142 end else begin
143     $display("PASS: OPMODE_reg cleared by RSTOPMODE");
144 end
145
146 if (dut.M_reg !== 36'h0) begin
147     $display("ERROR: M_reg not cleared by RSTM (0x%h)", dut.M_reg);
148     fail_count = fail_count + 1;
149 end else begin
150     $display("PASS: M_reg cleared by RSTM");
151 end
152
153 if (dut.P_reg !== 48'h0) begin
154     $display("ERROR: P_reg not cleared by RSTP (0x%h)", dut.P_reg);
155     fail_count = fail_count + 1;
156 end else begin
157     $display("PASS: P_reg cleared by RSTP");
158 end
159

```

```

160 // Self-checking: verify all outputs are zero
161 if (M == 36'h0 && P == 48'h0 && CARRYOUT == 1'b0 &&
162     CARRYOUTF == 1'b0 && BCOUT == 18'h0 && PCOUT == 48'h0) begin
163     $display("PASS: Reset Operation - All outputs are zero");
164     pass_count = pass_count + 1;
165 end else begin
166     $display("FAIL: Reset Operation");
167     $display("      M=%h, P=%h, CARRYOUT=%b, CARRYOUTF=%b, BCOUT=%h, PCOUT=%h",
168         M, P, CARRYOUT, CARRYOUTF, BCOUT, PCOUT);
169     fail_count = fail_count + 1;
170 end
171
172 // Deassert all reset signals and assert all clock enable signals
173 RSTA = 0; RSTB = 0; RSTC = 0; RSTCARRYIN = 0;
174 RSTD = 0; RSTM = 0; RSTOPMODE = 0; RSTP = 0;
175 CEA = 1; CEB = 1; CEC = 1; CECARRYIN = 1;
176 CED = 1; CEM = 1; CEOPMODE = 1; CEP = 1;
177
178 test_num = test_num + 1;
179 end
180 endtask

```

2. DSP Path 1 Verification

```
183 // Task 2: Verify DSP Path 1
184 task test_dsp_path_1;
185     begin
186         $display("\nTest 2: DSP Path 1 - Pre-subtractor and Post-subtractor");
187         $display("=====");
188         $display("Path: Pre-subtractor -> Multiplier -> Post-subtractor");
189         $display("OPMODE = 8'b11011101 (Pre-sub, Mult output to X, C to Z, Post-sub)");
190
191         // Apply specified input values
192         A = 20;
193         B = 10;
194         C = 350;
195         D = 25;
196         OPMODE = 8'b11011101;
197
198         // Drive cascade inputs with arbitrary values
199         BCIN = $random;
200         PCIN = $random;
201         CARRYIN = $random;
202
203         // Wait for four negative clock edges (pipeline depth)
204         repeat(4) @(negedge CLK);
205
206         // Expected outputs according to specification
207         // Pre-adder: D - B = 25 - 10 = 15
208         // Multiplier: A * (D - B) = 20 * 15 = 300 (0x12c)
209         // Post-adder: C - (A * (D-B)) = 350 - 300 = 50 (0x32)
210         // BCOUT should be B after pipeline = 10 (0xf)
211
212         if (BCOUT == 18'hf && M == 36'h12c && P == 48'h32 &&
213             PCOUT == 48'h32 && CARRYOUT == 1'b0 && CARRYOUTF == 1'b0) begin
214             $display("PASS: DSP Path 1");
215             $display("      BCOUT=%h, M=%h, P=%h, PCOUT=%h, CARRYOUT=%b",
216                 BCOUT, M, P, PCOUT, CARRYOUT);
217             pass_count = pass_count + 1;
218         end else begin
219             $display("FAIL: DSP Path 1");
220             $display("      Expected: BCOUT=0xf, M=0x12c, P=0x32, PCOUT=0x32, CARRYOUT=0");
221             $display("      Actual:   BCOUT=%h, M=%h, P=%h, PCOUT=%h, CARRYOUT=%b",
222                 BCOUT, M, P, PCOUT, CARRYOUT);
223             fail_count = fail_count + 1;
224         end
225
226         test_num = test_num + 1;
227     end
228 endtask
229
```

3. DSP Path 2 Verification

```
231 // Task 3: Verify DSP Path 2
232 task test_dsp_path_2;
233 begin
234     $display("\nTest 3: DSP Path 2 - Pre-adder with zeros routing");
235     $display("=====");
236     $display("Path: Pre-adder -> Multiplier -> Zero output");
237     $display("OPMODE = 8'b00010000 (Pre-add, zeros to X and Z)");
238
239     // Apply specified input values
240     A = 20;
241     B = 10;
242     C = 350;
243     D = 25;
244     OPMODE = 8'b00010000;
245
246     // Drive cascade inputs with arbitrary values
247     BCIN = $random;
248     PCIN = $random;
249     CARRYIN = $random;
250
251     // Wait for three negative clock edges
252     repeat(3) @(negedge CLK);
253
254     // Expected outputs according to specification
255     // Pre-adder: D + B = 25 + 10 = 35 (0x23)
256     // Multiplier: A * (D + B) = 20 * 35 = 700 (0x2bc)
257     // Post-adder: 0 + 0 = 0 (zeros through both muxes)
258
259     if (BCOUT == 18'h23 && M == 36'h2bc && P == 48'h0 &&
260         PCOUT == 48'h0 && CARRYOUT == 1'b0 && CARRYOUTF == 1'b0) begin
261         $display("PASS: DSP Path 2");
262         $display("      BCOUT=%h, M=%h, P=%h, PCOUT=%h, CARRYOUT=%b",
263             BCOUT, M, P, PCOUT, CARRYOUT);
264         pass_count = pass_count + 1;
265     end else begin
266         $display("FAIL: DSP Path 2");
267         $display("      Expected: BCOUT=0x23, M=0x2bc, P=0x0, PCOUT=0x0, CARRYOUT=0");
268         $display("      Actual:   BCOUT=%h, M=%h, P=%h, PCOUT=%h, CARRYOUT=%b",
269             BCOUT, M, P, PCOUT, CARRYOUT);
270         fail_count = fail_count + 1;
271     end
272
273     test_num = test_num + 1;
274 end
275 endtask
```

4. DSP Path 3 Verification

```
277 // Task 4: Verify DSP Path 3
278 task test_dsp_path_3;
279     begin
280         $display("\nTest 4: DSP Path 3 - P feedback path");
281         $display("=====");
282         $display("Path: No pre-adder -> Multiplier -> P feedback");
283         $display("OPMODE = 8'b00001010 (No pre-add, P feedback to X and Z)");
284
285         // Store previous P and CARRYOUT values for comparison
286         previous_P = P;
287         previous_CARRYOUT = CARRYOUT;
288
289         // Apply specified input values
290         A = 20;
291         B = 10;
292         C = 350;
293         D = 25;
294         OPMODE = 8'b00001010;
295
296         // Drive cascade inputs with arbitrary values
297         BCIN = $random;
298         PCIN = $random;
299         CARRYIN = $random;
300
301         // Wait for three negative clock edges
302         repeat(3) @(negedge CLK);
303
304         // Expected outputs according to specification
305         // No pre-adder: B passes through = 10 (0xa)
306         // Multiplier: A * B = 20 * 10 = 200 (0xc8)
307         // Post-adder: P + P = 2*P (P feedback to both X and Z)
308
309         if (BCOUT == 18'ha && M == 36'hc8 && P == previous_P &&
310             PCOUT == P && CARRYOUT == previous_CARRYOUT && CARRYOUTF == previous_CARRYOUT) begin
311             $display("PASS: DSP Path 3");
312             $display("      BCOUT=%h, M=%h, P=%h (previous), PCOUT=%h, CARRYOUT=%b",
313                 BCOUT, M, P, PCOUT, CARRYOUT);
314             pass_count = pass_count + 1;
315         end else begin
316             $display("FAIL: DSP Path 3");
317             $display("      Expected: BCOUT=0xa, M=0xc8, P=previous_P, PCOUT=P, CARRYOUT=previous");
318             $display("      Actual:   BCOUT=%h, M=%h, P=%h, PCOUT=%h, CARRYOUT=%b",
319                 BCOUT, M, P, PCOUT, CARRYOUT);
320             $display("      Previous: P=%h, CARRYOUT=%b", previous_P, previous_CARRYOUT);
321             fail_count = fail_count + 1;
322         end
323
324         test_num = test_num + 1;
325     end
326 endtask
```

4. DSP Path 4 Verification

```
328 // Task 5: Verify DSP Path 4
329 task test_dsp_path_4;
330 begin
331     $display("\nTest 5: DSP Path 4 - Concatenation and subtraction");
332     $display("=====");
333     $display("Path: No pre-adder -> Multiplier -> Concatenation subtraction");
334     $display("OPMODE = 8'b10100111 (No pre-add, D:A:B to X, PCIN to Z, subtract)");
335
336     // Apply specified input values
337     A = 5;
338     B = 6;
339     C = 350;
340     D = 25;
341     PCIN = 3000;
342     OPMODE = 8'b10100111;
343
344     // Drive remaining cascade inputs with arbitrary values
345     BCIN = $random;
346     CARRYIN = $random;
347
348     // Wait for three negative clock edges
349     repeat(3) @(negedge CLK);
350
351     // Expected outputs according to specification
352     // No pre-adder: B passes through = 6
353     // Multiplier: A * B = 5 * 6 = 30 (0x1e)
354     // Concatenation: {D[11:0], A, B} = {25[11:0], 5, 6} = {0x019, 0x00005, 0x00006}
355     // Post-subtractor: PCIN - {D[11:0], A, B} = 3000 - concatenated_value
356     // Expected: BCOUT=0x6, M=0x1e, P=PCOUT=0xfe6fffec0bb1, CARRYOUT=1
357
358     if (BCOUT == 18'h6 && M == 36'h1e && P == 48'hfe6fffec0bb1 &&
359         PCOUT == 48'hfe6fffec0bb1 && CARRYOUT == 1'b1 && CARRYOUTF == 1'b1) begin
360         $display("PASS: DSP Path 4");
361         $display("      BCOUT=%h, M=%h, P=%h, PCOUT=%h, CARRYOUT=%b",
362             BCOUT, M, P, PCOUT, CARRYOUT);
363         pass_count = pass_count + 1;
364     end else begin
365         $display("FAIL: DSP Path 4");
366         $display("      Expected: BCOUT=0x6, M=0x1e, P=0xfe6fffec0bb1, PCOUT=0xfe6fffec0bb1, CARRYOUT=1");
367         $display("      Actual:   BCOUT=%h, M=%h, P=%h, PCOUT=%h, CARRYOUT=%b",
368             BCOUT, M, P, PCOUT, CARRYOUT);
369         fail_count = fail_count + 1;
370     end
371
372     test_num = test_num + 1;
373 end
374 endtask
375
```

5. Task Execution and Test Summary

```
376 // Main test execution
377 initial begin
378     $display("Starting DSP48A1 Testbench - Specification Compliant");
379     $display("=====");
380
381     // Test sequence as per specification
382     test_reset_operation();
383     test_dsp_path_1();
384     test_dsp_path_2();
385     test_dsp_path_3();
386     test_dsp_path_4();
387
388     // Print final results
389     print_test_summary();
390     $stop;
391 end
392
393 // Task to print test summary
394 task print_test_summary;
395     begin
396         $display("\n=====");
397         $display("Test Summary:");
398         $display("Total Tests: %0d", test_num);
399         $display("Passed:      %0d", pass_count);
400         $display("Failed:      %0d", fail_count);
401         $display("=====");
402
403         if (fail_count == 0) begin
404             $display("ALL TESTS PASSED!");
405         end else begin
406             $display("SOME TESTS FAILED!");
407         end
408     end
409 endtask
410
411 endmodule
```

Third Section: DO File



```
1  vlib work
2  vlog DSP48A1.v DSP48A1_tb.v
3  vsim -voptargs=+acc work.DSP48A1_tb
4  add wave *
5  run -all
6
```

Fourth: Questa Sim

Starting DSP48A1 Testbench - Specification Compliant

Test 1: Reset Operation Verification

Loading test data into internal registers...

Internal registers BEFORE reset:

Al_reg = 0x3ffff (AOREG=0, only Al exists)
Bl_reg = 0x2aaab (BOREG=0, only Bl exists)
C_reg = 0xaaaaaaaaaaaa
D_reg = 0x15555
OPMODE_reg = 0xff
M_reg = 0xaaaa95555, P_reg = 0xxxxxxxxxxxxx

Internal registers AFTER reset:

Al_reg = 0x00000
Bl_reg = 0x00000
C_reg = 0x000000000000
D_reg = 0x00000
OPMODE_reg = 0x00
M_reg = 0x0000000000, P_reg = 0x000000000000

PASS: Al_reg cleared by RSTA

PASS: Bl_reg cleared by RSTB

PASS: C_reg cleared by RSTC

PASS: D_reg cleared by RSTD

PASS: OPMODE_reg cleared by RSTOPMODE

PASS: M_reg cleared by RSTM

PASS: P_reg cleared by RSTP

PASS: Reset Operation - All outputs are zero

Test 2: DSP Path 1 - Pre-subtractor and Post-subtractor

=====

Path: Pre-subtractor -> Multiplier -> Post-subtractor

OPMODE = 8'b11011101 (Pre-sub, Mult output to X, C to Z, Post-sub)

PASS: DSP Path 1

BCOUT=0000f, M=00000012c, P=000000000032, PCOUT=000000000032, CARRYOUT=0

#

Test 3: DSP Path 2 - Pre-adder with zeros routing

=====

Path: Pre-adder -> Multiplier -> Zero output

OPMODE = 8'b00010000 (Pre-add, zeros to X and Z)

PASS: DSP Path 2

BCOUT=00023, M=0000002bc, P=000000000000, PCOUT=000000000000, CARRYOUT=0

#

Test 4: DSP Path 3 - P feedback path

=====

Path: No pre-adder -> Multiplier -> P feedback

OPMODE = 8'b00001010 (No pre-add, P feedback to X and Z)

PASS: DSP Path 3

BCOUT=0000a, M=0000000c8, P=000000000000 (previous), PCOUT=000000000000, CARRYOUT=0

#

Test 5: DSP Path 4 - Concatenation and subtraction

=====

Path: No pre-adder -> Multiplier -> Concatenation subtraction

OPMODE = 8'b10100111 (No pre-add, D:A:B to X, PCIN to Z, subtract)

PASS: DSP Path 4

BCOUT=00006, M=00000001e, P=fe6fffec0bb1, PCOUT=fe6fffec0bb1, CARRYOUT=1

#

=====

Test Summary:

Total Tests: 5

Passed: 5

Failed: 0

=====

ALL TESTS PASSED!

** Note: \$stop : DSP48A1_tb.v(390)



● Fifth: Constraint File

Before Implementation Step in Vivado

```
## This file is a general .xdc for the Basys3 rev B board
## To use it in a project:
## - uncomment the lines corresponding to used pins
## - rename the used ports (in each line, after get_ports) according to the top level signal names in the project

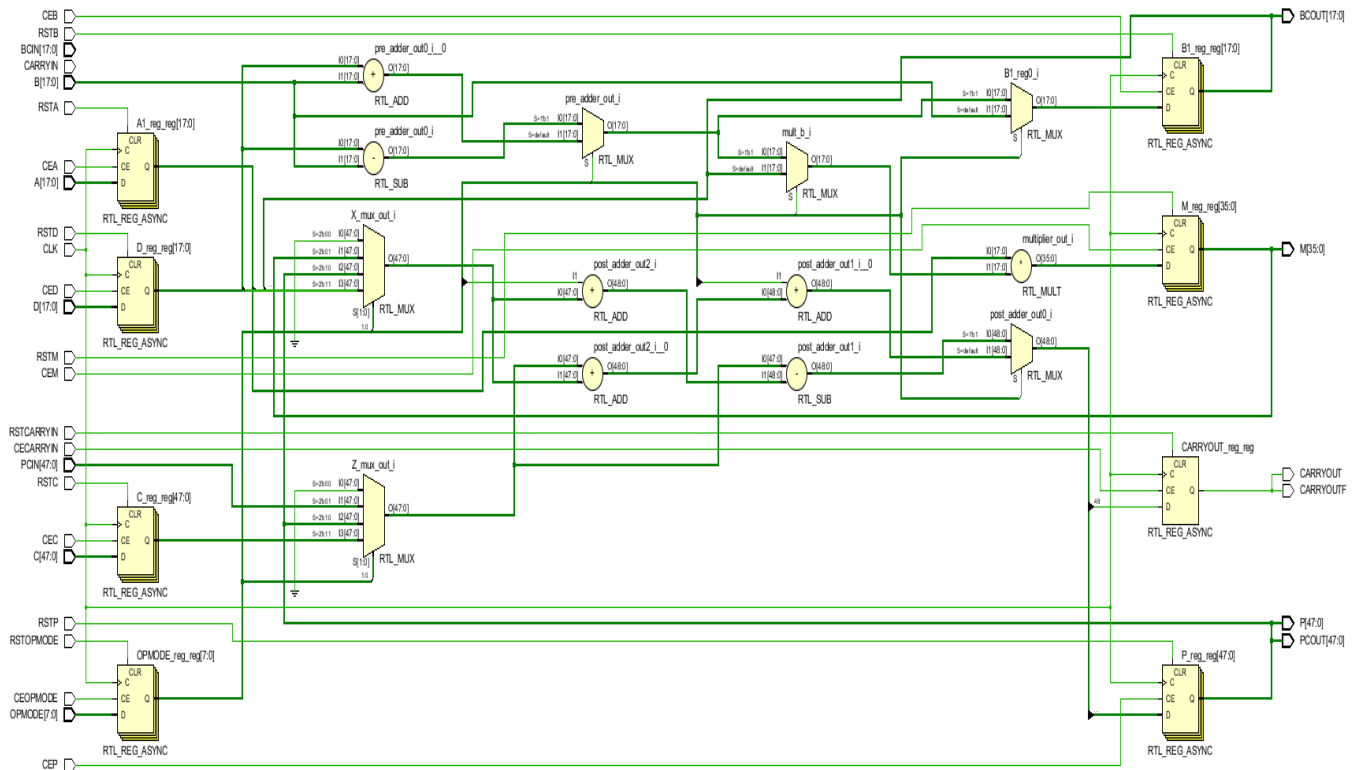
## Clock signal
set_property PACKAGE_PIN W5 [get_ports CLK]
set_property IOSTANDARD LVCMOS33 [get_ports CLK]
create_clock -period 10.000 -name sys_clk_pin -waveform {0.000 5.000} -add [get_ports CLK]
```

After Implementation Step in Vivado

```
create_debug_core u_ila_0_ila
set_property ALL_PROBE_NAME_MD true [get_debug_cores u_ila_0]
set_property ALL_PROBE_NAME_MD_CNT 1 [get_debug_cores u_ila_0]
set_property C_ADV_TRIGGER false [get_debug_cores u_ila_0]
set_property C_DATA_DEPTH 1024 [get_debug_cores u_ila_0]
set_property C_EN_STB_QUAL false [get_debug_cores u_ila_0]
set_property C_INPUT_PIPE_STAGES 0 [get_debug_cores u_ila_0]
set_property C_TRIGGER_EN false [get_debug_cores u_ila_0]
set_property C_TRIGGER_EN false [get_debug_cores u_ila_0]
set_property port_width 1 [get_debug_ports u_ila_0/clock]
connect_debug_port u_ila_0/clock [get_nets [list CLK_IBUF_BUF5]]
set_property FROME_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe0]
set_property port_width 19 [get_debug_ports u_ila_0/probe0]
connect_debug_port u_ila_0/probe0 [get_nets [list (A_IBUF[0]) (A_IBUF[1]) (A_IBUF[2]) (A_IBUF[3]) (A_IBUF[4]) (A_IBUF[5]) (A_IBUF[6]) (A_IBUF[7]) (A_IBUF[8]) (A_IBUF[9]) (A_IBUF[10]) (A_IBUF[11]) (A_IBUF[12]) (A_IBUF[13]) (A_IBUF[14]) (A_IBUF[15]) (A_IBUF[16]) (A_IBUF[17])]]]
create_debug_port u_ila_0/probe
set_property FROME_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe1]
set_property port_width 10 [get_debug_ports u_ila_0/probe1]
connect_debug_port u_ila_0/probe1 [get_nets [list (BCOUT_OBUF[0]) (BCOUT_OBUF[1]) (BCOUT_OBUF[2]) (BCOUT_OBUF[3]) (BCOUT_OBUF[4]) (BCOUT_OBUF[5]) (BCOUT_OBUF[6]) (BCOUT_OBUF[7]) (BCOUT_OBUF[8]) (BCOUT_OBUF[9]) (BCOUT_OBUF[10]) (BCOUT_OBUF[11]) (BCOUT_OBUF[12]) (BCOUT_OBUF[13]) (BCOUT_OBUF[14]) (BCOUT_OBUF[15])]]]
create_debug_port u_ila_0/probe
set_property FROME_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe2]
set_property port_width 36 [get_debug_ports u_ila_0/probe2]
connect_debug_port u_ila_0/probe2 [get_nets [list (H_OBUF[0]) (H_OBUF[1]) (H_OBUF[2]) (H_OBUF[3]) (H_OBUF[4]) (H_OBUF[5]) (H_OBUF[6]) (H_OBUF[7]) (H_OBUF[8]) (H_OBUF[9]) (H_OBUF[10]) (H_OBUF[11]) (H_OBUF[12]) (H_OBUF[13]) (H_OBUF[14]) (H_OBUF[15]) (H_OBUF[16]) (H_OBUF[17]) (H_OBUF[18]) (H_OBUF[19]) (H_OBUF[20])]]]
create_debug_port u_ila_0/probe
set_property FROME_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe3]
set_property port_width 8 [get_debug_ports u_ila_0/probe3]
connect_debug_port u_ila_0/probe3 [get_nets [list (ORNODE_IBUF[0]) (ORNODE_IBUF[1]) (ORNODE_IBUF[2]) (ORNODE_IBUF[3]) (ORNODE_IBUF[4]) (ORNODE_IBUF[5]) (ORNODE_IBUF[6]) (ORNODE_IBUF[7])]]]
create_debug_port u_ila_0/probe
set_property FROME_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe4]
set_property port_width 48 [get_debug_ports u_ila_0/probe4]
connect_debug_port u_ila_0/probe4 [get_nets [list (PCOUT_OBUF[0]) (PCOUT_OBUF[1]) (PCOUT_OBUF[2]) (PCOUT_OBUF[3]) (PCOUT_OBUF[4]) (PCOUT_OBUF[5]) (PCOUT_OBUF[6]) (PCOUT_OBUF[7]) (PCOUT_OBUF[8]) (PCOUT_OBUF[9]) (PCOUT_OBUF[10]) (PCOUT_OBUF[11]) (PCOUT_OBUF[12]) (PCOUT_OBUF[13]) (PCOUT_OBUF[14]) (PCOUT_OBUF[15])]]]
create_debug_port u_ila_0/probe
set_property FROME_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe5]
set_property port_width 48 [get_debug_ports u_ila_0/probe5]
connect_debug_port u_ila_0/probe5 [get_nets [list (C_IBUF[0]) (C_IBUF[1]) (C_IBUF[2]) (C_IBUF[3]) (C_IBUF[4]) (C_IBUF[5]) (C_IBUF[6]) (C_IBUF[7]) (C_IBUF[8]) (C_IBUF[9]) (C_IBUF[10]) (C_IBUF[11]) (C_IBUF[12]) (C_IBUF[13]) (C_IBUF[14]) (C_IBUF[15]) (C_IBUF[16]) (C_IBUF[17]) (C_IBUF[18]) (C_IBUF[19]) (C_IBUF[20])]]]
create_debug_port u_ila_0/probe
set_property FROME_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe6]
set_property port_width 19 [get_debug_ports u_ila_0/probe6]
connect_debug_port u_ila_0/probe6 [get_nets [list (D_IBUF[0]) (D_IBUF[1]) (D_IBUF[2]) (D_IBUF[3]) (D_IBUF[4]) (D_IBUF[5]) (D_IBUF[6]) (D_IBUF[7]) (D_IBUF[8]) (D_IBUF[9]) (D_IBUF[10]) (D_IBUF[11]) (D_IBUF[12]) (D_IBUF[13]) (D_IBUF[14]) (D_IBUF[15]) (D_IBUF[16]) (D_IBUF[17])]]]
create_debug_port u_ila_0/probe
set_property FROME_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe7]
set_property port_width 10 [get_debug_ports u_ila_0/probe7]
connect_debug_port u_ila_0/probe7 [get_nets [list (B_IBUF[0]) (B_IBUF[1]) (B_IBUF[2]) (B_IBUF[3]) (B_IBUF[4]) (B_IBUF[5]) (B_IBUF[6]) (B_IBUF[7]) (B_IBUF[8]) (B_IBUF[9]) (B_IBUF[10]) (B_IBUF[11]) (B_IBUF[12]) (B_IBUF[13]) (B_IBUF[14]) (B_IBUF[15]) (B_IBUF[16]) (B_IBUF[17])]]]
create_debug_port u_ila_0/probe
set_property FROME_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe8]
set_property port_width 48 [get_debug_ports u_ila_0/probe8]
connect_debug_port u_ila_0/probe8 [get_nets [list (PCIN_IBUF[0]) (PCIN_IBUF[1]) (PCIN_IBUF[2]) (PCIN_IBUF[3]) (PCIN_IBUF[4]) (PCIN_IBUF[5]) (PCIN_IBUF[6]) (PCIN_IBUF[7]) (PCIN_IBUF[8]) (PCIN_IBUF[9]) (PCIN_IBUF[10]) (PCIN_IBUF[11]) (PCIN_IBUF[12]) (PCIN_IBUF[13]) (PCIN_IBUF[14]) (PCIN_IBUF[15]) (PCIN_IBUF[16])]]]
create_debug_port u_ila_0/probe
set_property FROME_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe9]
set_property port_width 1 [get_debug_ports u_ila_0/probe9]
connect_debug_port u_ila_0/probe9 [get_nets [list CARRYOUTT_OBUF]]
create_debug_port u_ila_0/probe
set_property FROME_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe10]
set_property port_width 1 [get_debug_ports u_ila_0/probe10]
connect_debug_port u_ila_0/probe10 [get_nets [list CEA_IBUF]]
create_debug_port u_ila_0/probe
set_property FROME_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe11]
set_property port_width 1 [get_debug_ports u_ila_0/probe11]
connect_debug_port u_ila_0/probe11 [get_nets [list CEB_IBUF]]
create_debug_port u_ila_0/probe
set_property FROME_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe12]
set_property port_width 1 [get_debug_ports u_ila_0/probe12]
connect_debug_port u_ila_0/probe12 [get_nets [list CKC_IBUF]]
create_debug_port u_ila_0/probe
set_property FROME_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe13]
set_property port_width 1 [get_debug_ports u_ila_0/probe13]
connect_debug_port u_ila_0/probe13 [get_nets [list CECARRYIN_IBUF]]
create_debug_port u_ila_0/probe
```

Sixth: Elaboration

SCHEMATIC



Elaborated Design Warnings

Tcl Console Messages Log Reports Design Runs

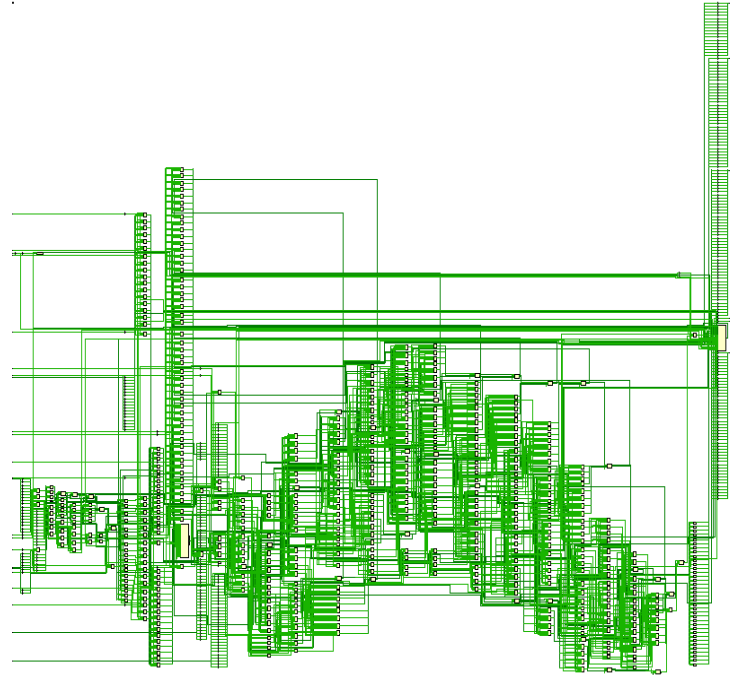
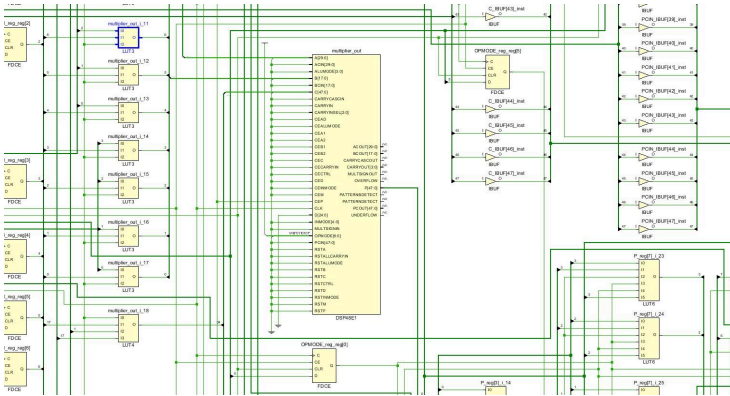
Warning (67) Info (273) Status (523) Show All

Elaborated Design (21 warnings)

- General Messages (21 warnings)
 - [Synth 8-2490] overwriting previous definition of module DSP48A1 [DSP48A1.v:1]
 - [Synth 8-6014] Unused sequential element CARRYIN_reg_reg was removed. [DSP48A1.v:291]
 - [Synth 8-3331] design DSP48A1 has unconnected port CARRYIN (18 more like this)
- Synthesis (41 warnings)
- Implementation (1 warning)
 - Route Design (1 warning)
 - [Timing 38-436] There are set_bus_skew constraint(s) in this design. Please run report_bus_skew to ensure that bus skew requirements are met.
- Implemented Design (2 warnings)
- Simulation (1 warning)
 - sim_1 (1 warning)
 - [XSIM 43-4100] "D:\Hardware\Projects\Project_1\project_1\project_1.sim\sim_1\behav\sim\gbl.v" Line 6. Module gbl has a timescale but at least one module in design doesn't have timescale.

Seventh: Synthesis

SCHEMATIC



Synthesis Design Warnings

Tcl Console Messages x Log Reports Design Runs Timing Debug

Warning (67) Info (273) Status (523) Show All

> Elaborated Design (21 warnings)

▼ Synthesis (41 warnings)

- [Synth 8-2490] overwriting previous definition of module DSP48A1 [DSP48A1.v:1]
- [Synth 8-6014] Unused sequential element CARRYIN_reg_reg was removed. [DSP48A1.v:291]
- > [Synth 8-3331] design DSP48A1 has unconnected port CARRYIN (37 more like this)
- [Constraints 18-5210] No constraint will be written out.

Timing & Utilization Report

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 4.201 ns	Worst Hold Slack (WHS): 0.231 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 120	Total Number of Endpoints: 120	Total Number of Endpoints: 162

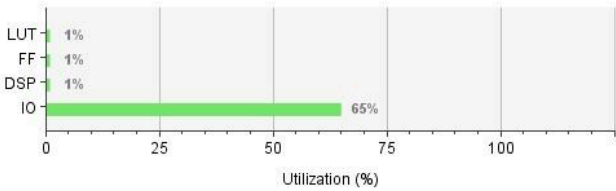
All user specified timing constraints are met.



Summary

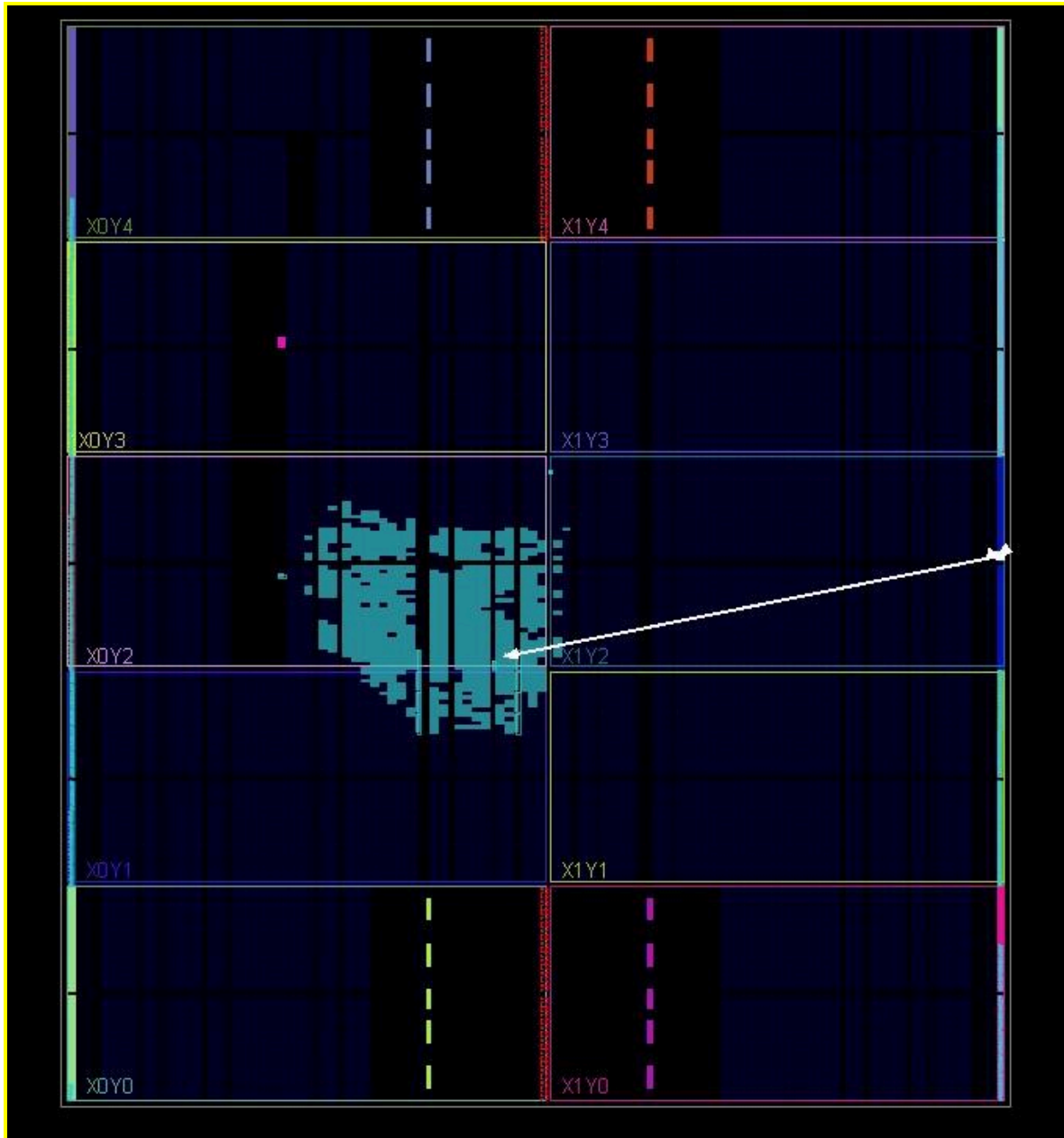
- Hierarchy
- Summary
- ▼ Slice Logic
 - ▼ Slice LUTs (<1%)
 - LUT as Logic (<1%)
 - ▼ Slice Registers (<1%)
 - Register as Flip Flop (<1%)
 - Memory
 - ▼ DSP
 - ▼ DSPs (<1%)
 - DSP48E1 only
 - ▼ IO and GT Specific
 - ▼ Bonded IOB (65%)
 - IOB Master Pads
 - ▼ Clocking
 - BUFGCTRL (3%)
 - Specific Feature
 - Primitives
 - ▼ Black Boxes
 - u_ila_0_CV
 - dbg_hub_CV
 - Instantiated Netlists

Resource	Utilization	Available	Utilization %
LUT	368	134600	0.27
FF	160	269200	0.06
DSP	1	740	0.14
IO	327	500	65.40



Eighth: Implementation

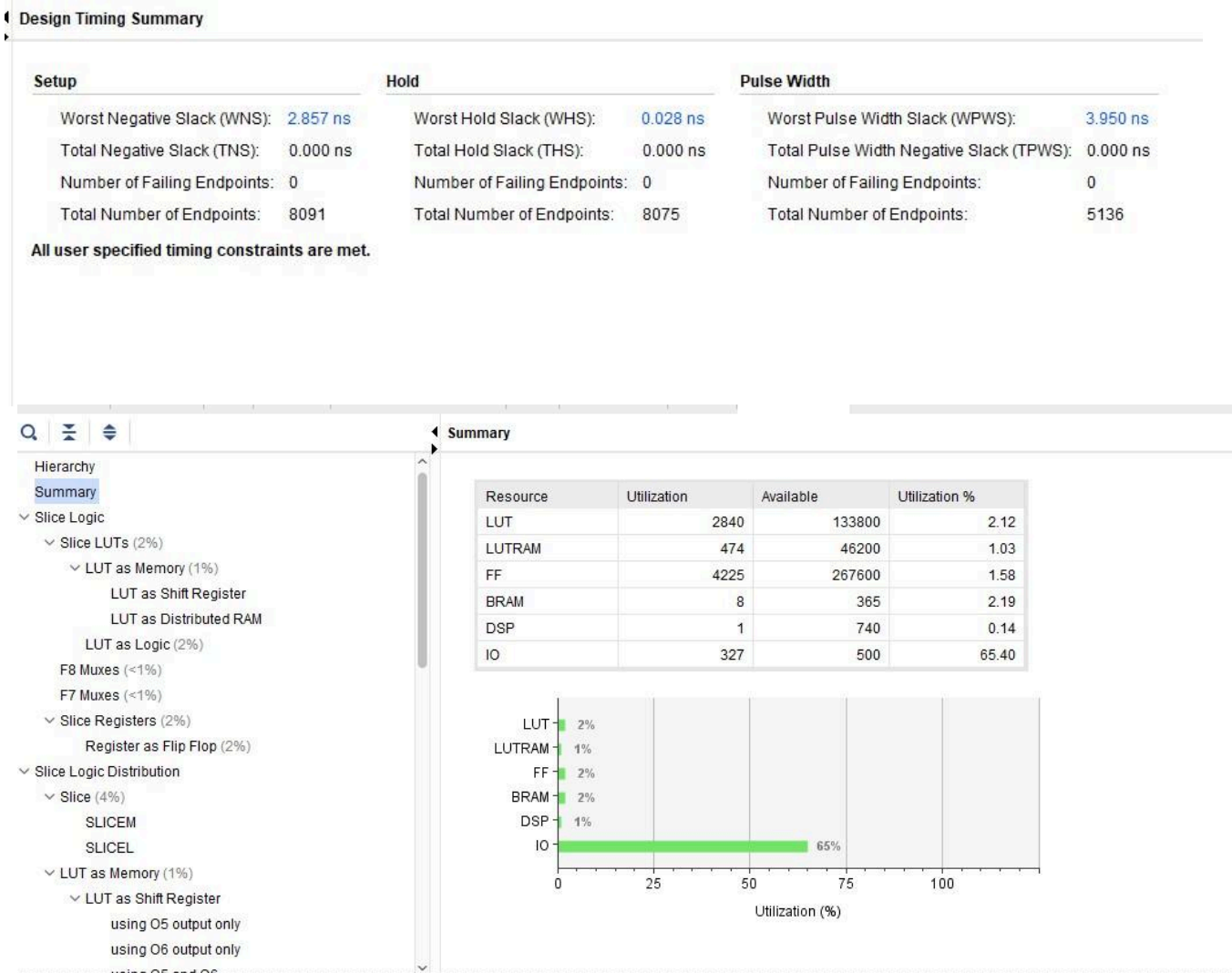
SCHEMATIC



Synthesis Design Warnings

- Implementation (1 warning)
 - Route Design (1 warning)
 - [Timing 38-436] There are set_bus_skew constraint(s) in this design. Please run report_bus_skew to ensure that bus skew requirements are met.
- Implemented Design (2 warnings)
 - General Messages (2 warnings)
 - [Timing 38-436] There are set_bus_skew constraint(s) in this design. Please run report_bus_skew to ensure that bus skew requirements are met. (1 more like this)
- Simulation (1 warning)

Timing & Utilization Report



● Ninth Section: LINT Snippets

Flow Navigator

Compile Design

This step will compile the design

Compile Errors/Warnings

Status : Compilation Completed

Elapsed Time : 488ms

Compile Design

Status	Command	Results
✓	vlog D:/Hardware/Projects/Project_1/DSP48A1.v -work work	0 Warning, 0 Error

Next Step Add Directives

Compilation Progress 100%

Details

Check : inferred_blackbox

Module : DSP48A1

Reference : D:/Hardware/Projects/Project_1/DSP48A1.v_1

Message : Module is inferred as blackbox. Module DSP48A1, File D:/Hardware/Projects/Project_1/DSP48A1.v, Line 1.

Status : unassigned

Owner : unassigned

```
1 module DSP48A1 (#
2     parameter AREG = 0,
3     parameter AIREG = 1,
4     parameter BIREG = 0,
5     parameter BIREG = 1,
6     parameter CREG = 1,
7     parameter DREG = 1,
8     parameter MREG = 1,
9     parameter PREG = 1,
10    parameter CARRYINREG = 1,
11    parameter CARRYOUTREG = 1,
12    parameter OPMODEREG = 1,
13
14    // Configuration parameters
15    parameter CARRYINSEL = "OPMODE5", // "CARRYIN" or "OPMODE5"
16    parameter B_INPUT = "DIRECT", // "DIRECT" or "CASCADE"
17    parameter RSTTYPE = "SYNC", // "SYNC" or "ASYN"
18
19 )
20
21 // Clock
22 input CLK,
23
24 //Data Inputs
25
26 input [17:0] A,
27 input [17:0] B,
28 input [47:0] C,
29 input [17:0] D,
30 input CARRYIN,
31
32 // Control Inputs
33 input [7:0] OPMODE,
34
35 // Clock enables
36 input CEA,
37 input CEB,
38 input CEC,
39 input CECARRYIN,
40 input CED,
41 input CEM,
42 input CEOPMODE,
43 input CEP,
44
45 // Resets
46 input RSTA,
47 input RSTB,
48 input RSTC,
49 input RSTCARRYIN,
50 input RSTD,
51 input RSTM,
52 input RSTM,
53
54 module DSP48A1;
```

Lint Summary

(Type Search Text (Press Enter))

Name	Count
Resolved(verified, fixed, waived)	1
Warning	1
inferred_blackbox	1

Flow Navigator Details Design

Lint Checks

Filter: Type here

Severity Status Check Alias Message Module Category State Owner

Transcript Message Viewer Lint Checks Design Metrics Design Information Status History Lint Dashboard